

✓ YZTA 2026 Datathon

1. Kütüphanelerin Kurulması ve Import Edilmesi

```
!pip install catboost lightgbm xgboost shap optuna -q

import warnings
warnings.filterwarnings("ignore")

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import (
    KFold,
    cross_val_score
)

from sklearn.metrics import (
    mean_squared_error,
    mean_absolute_error,
    r2_score
)

from sklearn.preprocessing import (
    LabelEncoder
)

from sklearn.feature_selection import (
    VarianceThreshold
)

from sklearn.linear_model import (
    LinearRegression
)

from sklearn.ensemble import (
    RandomForestRegressor,
    ExtraTreesRegressor
)

from catboost import CatBoostRegressor
from lightgbm import LGBMRegressor
from xgboost import XGBRegressor

import shap
import optuna
```

97.1/97.1 MB 7.0 MB/s eta 0:00:00

2. Veri Setinin Yükleneşi

```
train = pd.read_csv("train.csv")
test = pd.read_csv("test_x.csv")

TARGET = "bilissel_performans_skoru"
```

3. Veri Setine Genel Bakış

3.1 Veri Boyutu İncelemesi

```
print(train.shape)
print(test.shape)
```

```
(56000, 24)
(24000, 23)
```

3.2 İlk Gözlemler

```
print(train.head())
```

```
   id  yas cinsiyet  meslek  vucut_kitle_indeksi  ulke \
0  1   34   Erkek  Saglik Personeli      31.470103    Cin
1  2   32   Kadin  Muhendis      30.981394    Amerika
2  3   39   Erkek  Ev Hanimi      21.533898    Spain
3  4   40   Kadin  Egitimci      23.310749  Yeni Zelanda
4  5   36   Kadin      NaN      NaN      Portekiz

   rem_yuzdesi  derin_uyku_yuzdesi  uykuya_dalma_suresi_dk \
0      14.431210      14.645436      27
1      21.771870      27.220360      20
2      18.178857      25.530104      33
3      21.438151      15.891188      21
4      25.468018      16.356738      21

   gecelik_uyanma_sayisi  ...  stres_skoru  gunluk_calisma_saati \
0              7  ...      9.922976      10.045274
1              4  ...      6.626400      6.319245
2              7  ...      6.093566      3.824463
3              2  ...      3.168185      4.597316
4              8  ...      7.198574      3.189120

   kronotip  ruh_sagligi_durumu  dinlenik_nabiz_bpm \
0 Sabah insani  Anksiyete ve depresyon      78
1 Gece insani  Saglikli      76
2      Notr  Depresyon      66
3 Gece insani  Saglikli      60
4      Notr  Anksiyete ve depresyon      74

   oda_sicakligi_celsius  hafta_sonu_uyku_farki_saat  mevsim  gun_tipi \
0      18.962436      -0.074140  Sonbahar-Kis  Hafta ici
1      21.225666      0.942672  Sonbahar-Kis  Hafta ici
2      18.482409      1.239886  Ilkbahar-Yaz  Hafta sonu
3      21.862235      0.727695  Sonbahar-Kis  Hafta sonu
4      19.223195      -0.223402  Sonbahar-Kis  Hafta ici

   bilissel_performans_skoru
0      0.136441
1      5.848312
2      6.828276
3      8.144649
4      0.431423
```

```
[5 rows x 24 columns]
```

3.3 Veri Tipleri Analizi

```
print(train.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 56000 entries, 0 to 55999
Data columns (total 24 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   id                                    56000 non-null  int64
 1   yas                                    56000 non-null  int64
 2   cinsiyet                             56000 non-null  object
 3   meslek                                54622 non-null  object
 4   vucut_kitle_indeksi                   54248 non-null  float64
 5   ulke                                  56000 non-null  object
 6   rem_yuzdesi                           56000 non-null  float64
 7   derin_uyku_yuzdesi                   56000 non-null  float64
 8   uykuya_dalma_suresi_dk                56000 non-null  int64
 9   gecelik_uyanma_sayisi                 56000 non-null  int64
10  uyku_onesi_kafein_mg                   54537 non-null  float64
11  uyku_onesi_ekran_suresi_dk             56000 non-null  int64
12  gunluk_adim_sayisi                     56000 non-null  int64
13  sekerleme_suresi_dk                   56000 non-null  int64
14  stres_skoru                           54285 non-null  float64
15  gunluk_calisma_saati                   56000 non-null  float64
16  kronotip                              54032 non-null  object
17  ruh_sagligi_durumu                     54904 non-null  object
18  dinlenik_nabiz_bpm                    56000 non-null  int64
19  oda_sicakligi_celsius                  56000 non-null  float64
20  hafta_sonu_uyku_farki_saat             56000 non-null  float64
21  mevsim                                 56000 non-null  object
22  gun_tipi                               56000 non-null  object
23  bilissel_performans_skoru              56000 non-null  float64
dtypes: float64(9), int64(8), object(7)
memory usage: 10.3+ MB
None
```

3.4 İstatistiksel Özet

```
print(train.describe().T)
```

	count	mean	std	min	\
id	56000.0	28000.500000	16165.951874	1.0	
yas	56000.0	34.714446	11.052508	18.0	
vucut_kitle_indeksi	54248.0	26.273651	4.495138	16.0	
rem_yuzdesi	56000.0	20.233500	3.442993	10.0	
derin_uyku_yuzdesi	56000.0	20.246063	4.300421	5.0	
uykuya_dalma_suresi_dk	56000.0	20.107536	8.063278	1.0	
gecelik_uyanma_sayisi	56000.0	3.344929	2.034618	0.0	
uyku_onesi_kafein_mg	54537.0	42.110952	72.154159	0.0	
uyku_onesi_ekran_suresi_dk	56000.0	63.511446	44.590737	2.0	
gunluk_adim_sayisi	56000.0	7493.197554	3462.699361	500.0	
sekerleme_suresi_dk	56000.0	15.432875	21.104672	0.0	
stres_skoru	54285.0	5.741407	1.624985	1.0	
gunluk_calisma_saati	56000.0	7.156456	3.469731	0.0	
dinlenik_nabiz_bpm	56000.0	66.603214	7.353174	45.0	
oda_sicakligi_celsius	56000.0	20.531263	2.903317	15.0	
hafta_sonu_uyku_farki_saat	56000.0	1.199050	0.790899	-1.0	
bilissel_performans_skoru	56000.0	5.913096	2.231759	0.0	
	25%	50%	75%	max	
id	14000.750000	28000.500000	42000.250000	56000.0	
yas	26.000000	33.000000	42.000000	69.0	
vucut_kitle_indeksi	23.175279	26.225806	29.321503	45.0	
rem_yuzdesi	17.951724	20.302218	22.598180	30.0	
derin_uyku_yuzdesi	17.337866	20.292151	23.227117	30.0	
uykuya_dalma_suresi_dk	14.000000	20.000000	25.000000	58.0	
gecelik_uyanma_sayisi	2.000000	3.000000	5.000000	8.0	
uyku_onesi_kafein_mg	0.000000	3.000000	77.000000	400.0	
uyku_onesi_ekran_suresi_dk	30.000000	51.000000	84.000000	180.0	
gunluk_adim_sayisi	5042.000000	7426.000000	9876.000000	20000.0	
sekerleme_suresi_dk	0.000000	2.000000	30.000000	115.0	
stres_skoru	4.761271	5.824014	6.837436	10.0	
gunluk_calisma_saati	4.765942	7.360860	9.684531	18.0	
dinlenik_nabiz_bpm	62.000000	67.000000	72.000000	97.0	
oda_sicakligi_celsius	18.457240	20.487456	22.522152	28.0	
hafta_sonu_uyku_farki_saat	0.660030	1.202921	1.743940	3.0	
bilissel_performans_skoru	4.397431	6.032249	7.574980	10.0	

4. Eksik Veri Analizi

4.1 Eksik Değer Sayıları

```
missing_count = train.isnull().sum()
missing_count = missing_count[missing_count > 0]
print(missing_count)
```

```
meslek      1378
vucut_kitle_indeksi  1752
uyku_onesi_kafein_mg  1463
stres_skoru  1715
kronotip    1968
ruh_sagligi_durumu  1096
dtype: int64
```

Eksik Veri Oranları

```
missing_ratio = train.isnull().mean() * 100
missing_ratio = missing_ratio[missing_ratio > 0]
print(missing_ratio)
```

```
meslek      2.460714
vucut_kitle_indeksi  3.128571
uyku_onesi_kafein_mg  2.612500
stres_skoru  3.062500
kronotip    3.514286
ruh_sagligi_durumu  1.957143
dtype: float64
```

5. Hedef Değişken Analizi

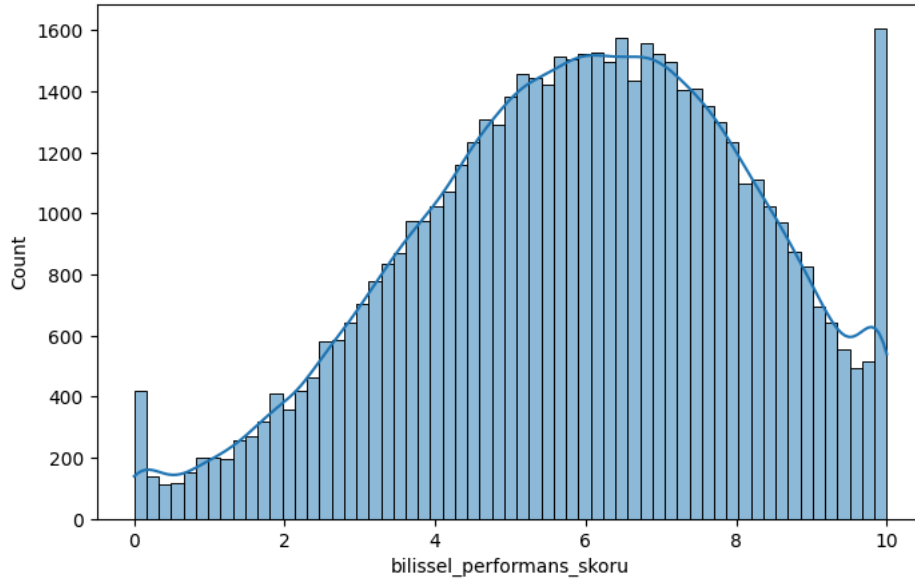
5.1 Target Distribution

```
plt.figure(figsize=(8,5))

sns.histplot(
```

```
train[TARGET],
kde=True
)
```

<Axes: xlabel='bilissel_performans_skoru', ylabel='Count'>

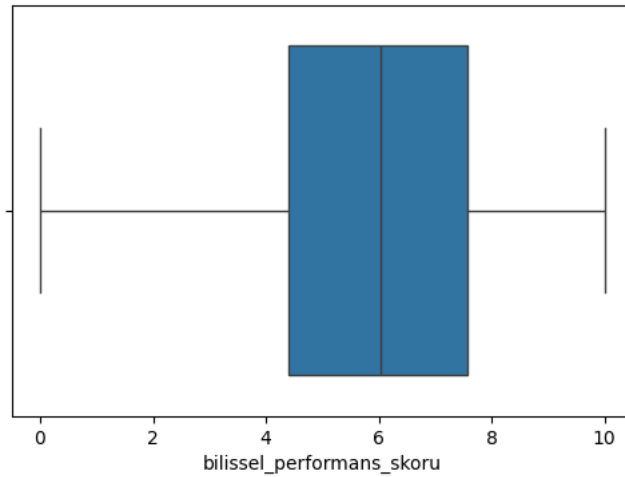


5.2 Target Boxplot Analizi

```
plt.figure(figsize=(6,4))

sns.boxplot(
    x=train[TARGET]
)

plt.show()
```



5.3 Çarpıklık (Skewness) Analizi

```
print("Target Skewness:", train[TARGET].skew())

Target Skewness: -0.2885088444537524
```

6. Sayısal Değişken Analizi

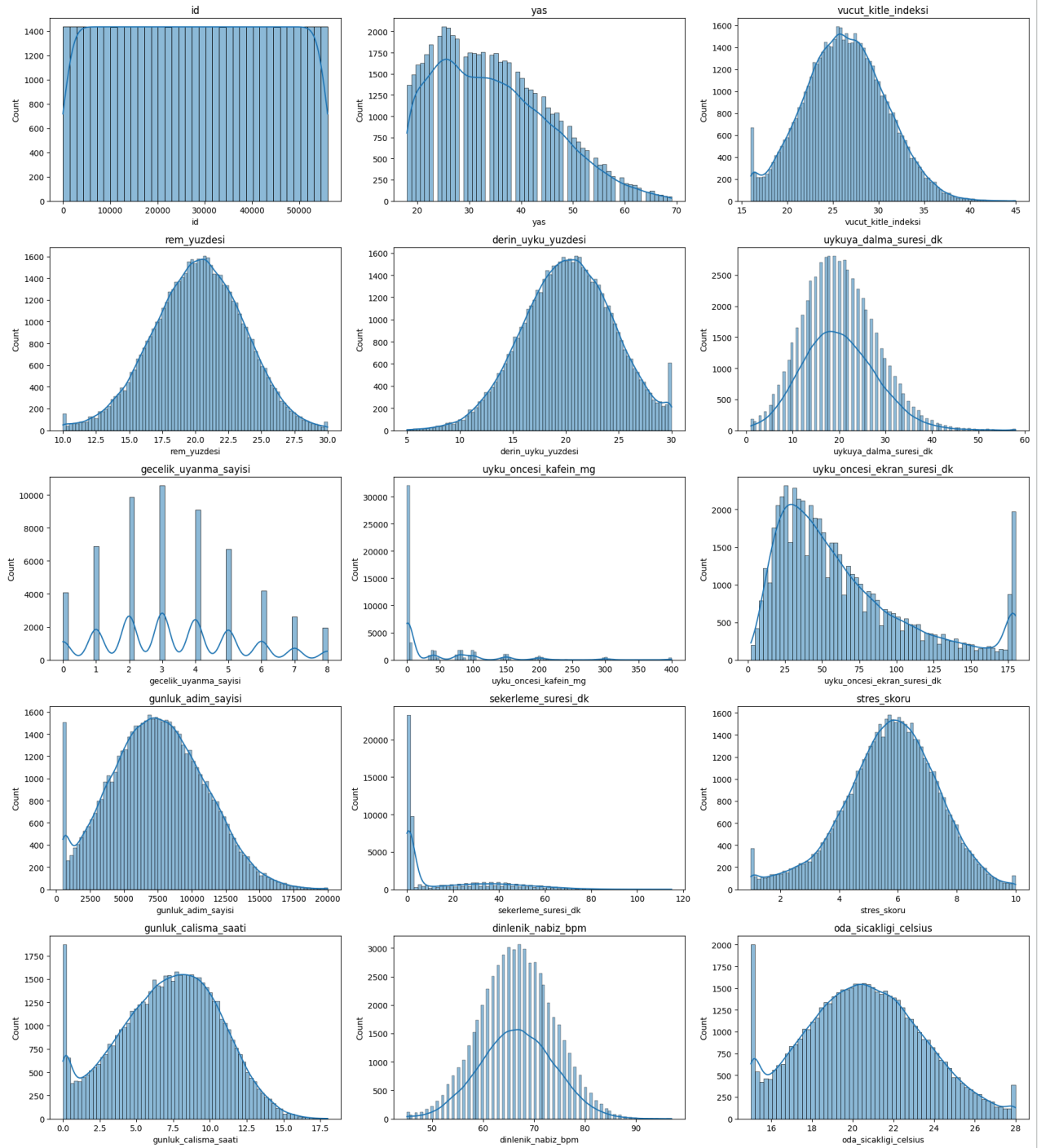
6.1 Histogram Analizleri

```
num_cols = train.select_dtypes(include=["int64", "float64"]).columns.tolist()
num_cols = [col for col in num_cols if col != TARGET]

fig, axes = plt.subplots(5, 3, figsize=(18, 20))
```

```
for col, ax in zip(num_cols, axes.ravel()):
    sns.histplot(train[col], kde=True, ax=ax)
    ax.set_title(col)
```

```
plt.tight_layout()
plt.show()
```



6.2 Boxplot Analizleri

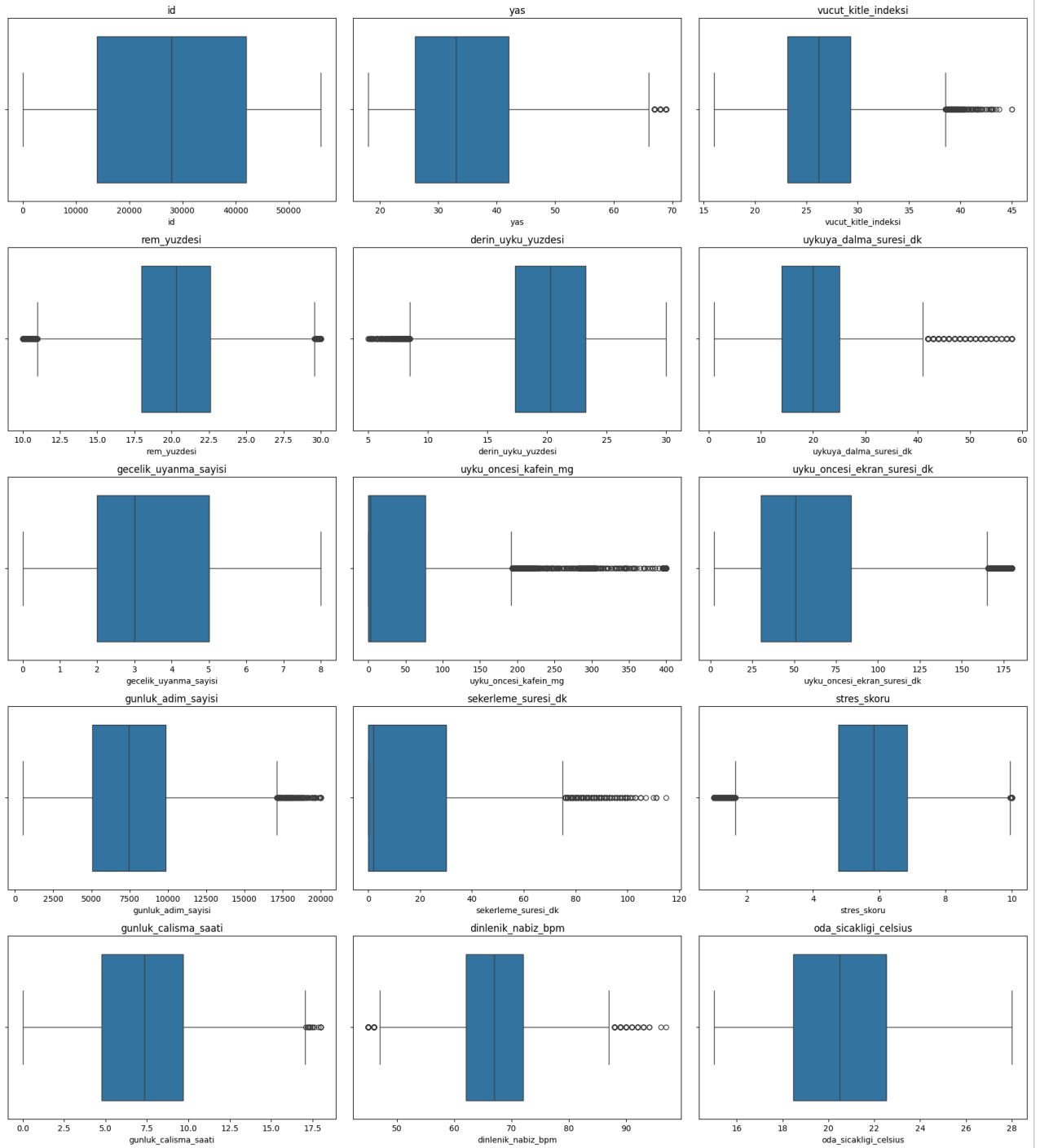
```
fig, axes = plt.subplots(5, 3, figsize=(18, 20))

for col, ax in zip(num_cols, axes.ravel()):
    sns.boxplot(x=train[col], ax=ax)
```

```
ax.set_title(col)
```

```
plt.tight_layout()
```

```
plt.show()
```



6.3 Sayısal Değişken Çarpıklıkları

```
print(  
    train[num_cols]  
    .skew()  
    .sort_values(ascending=False)  
)
```

```

uyku_onesi_kafein_mg      2.294817
sekerleme_suresi_dk      1.227650
uyku_onesi_ekran_suresi_dk 1.129793
yas                      0.552386
uykuya_dalma_suresi_dk   0.436116
gecelik_uyanma_sayisi    0.356814
gunluk_adim_sayisi       0.151099
oda_sicakligi_celsius    0.138846
vucut_kitle_indeksi      0.077956
id                       0.000000
dinlenik_nabiz_bpm      -0.001181
hafta_sonu_uyku_farki_saat -0.055557
derin_uyku_yuzdesi      -0.111748
rem_yuzdesi              -0.113536
gunluk_calisma_saati     -0.152106
stres_skoru              -0.341189
dtype: float64

```

7. Kategorik Değişken Analizi

7.1 Frekans Analizi

```

cat_cols = train.select_dtypes(
    include="object"
).columns.tolist()

for col in cat_cols:

    print("\n", col)

    print(
        train[col]
        .value_counts()
    )

```

```

Saglik Personeli      9842
Ogrenci               8132
Muhendis              6533
Yonetici              4454
Egitimci              4395
Satis ve Pazarlama Calisani 3899
Serbest Calisan       3812
Lojistik Calisani     3811
Emekli                3778
Ev Hanimi             3224
Lawyer                2742
Name: count, dtype: int64

```

```

ulke
ulke
Ingiltere      11324
Cin            9989
Guney Kore    4459
Amerika        4447
Arjantin       3973
Fransa         3440
Ispanya        2823
Isvec          2798
Yeni Zelanda   2777
Portekiz       2204
South Korea    2189
Mexico         2165
Netherlands    1707
Spain          1139
Sweden         566
Name: count, dtype: int64

```

```

kronotip
kronotip
Notr           24594
Gece insani    15938
Sabah insani   13500
Name: count, dtype: int64

```

```
ruh_sagligi_durumu
```

```

Ilkbahar-Yaz      28143
Sonbahar-Kis      27857
Name: count, dtype: int64

gun_tipi
gun_tipi
Hafta ici         40052
Hafta sonu        15948
Name: count, dtype: int64

```

7.2 Target Ortalama Analizi

```

for col in cat_cols:

    print(
        train.groupby(col)[TARGET]
        .mean()
        .sort_values(ascending=False)
    )

```

```

cinsiyet
Erkek      5.913804
Kadin      5.912420
Name: bilissel_performans_skoru, dtype: float64
meslek
Emekli                        7.935044
Ev Hanimi                     7.055969
Serbest Calisan               6.985534
Egitimci                      6.343725
Muhendis                     6.223087
Satis ve Pazarlama Calisani   6.010286
Yonetici                      5.729820
Ogrenci                       5.535248
Lojistik Calisani             5.119135
Saglik Personeli              4.930272
Lawyer                        4.719782
Name: bilissel_performans_skoru, dtype: float64
ulke
Spain          5.990125
Isvec          5.982263
Netherlands    5.974815
Mexico         5.951125
Ispanya        5.924855
Yeni Zelanda   5.920415
Ingiltere      5.915436
Cin            5.912783
Guney Kore     5.909655
Portekiz       5.905152
Arjantin       5.888813
Amerika        5.884016
South Korea    5.882008
Sweden         5.874886
Fransa         5.855670
Name: bilissel_performans_skoru, dtype: float64
kronotip
Sabah insani    5.995613
Notr            5.934548
Gece insani     5.797330
Name: bilissel_performans_skoru, dtype: float64
ruh_sagligi_durumu
Saglikli        6.491822
Anksiyete       5.012262
Depresyon       4.394400
Anksiyete ve depresyon  3.590186
Name: bilissel_performans_skoru, dtype: float64
mevsim
Sonbahar-Kis    5.915958
Ilkbahar-Yaz    5.910262
Name: bilissel_performans_skoru, dtype: float64
gun_tipi
Hafta sonu      6.993307
Hafta ici       5.482975
Name: bilissel_performans_skoru, dtype: float64

```

8. Korelasyon Analizi

8.1 Heatmap Görselleştirmesi

```

corr_matrix = train.corr(numeric_only=True).abs()
plt.figure(figsize=(16,12))

sns.heatmap(
    corr_matrix,
    cmap="coolwarm",

```

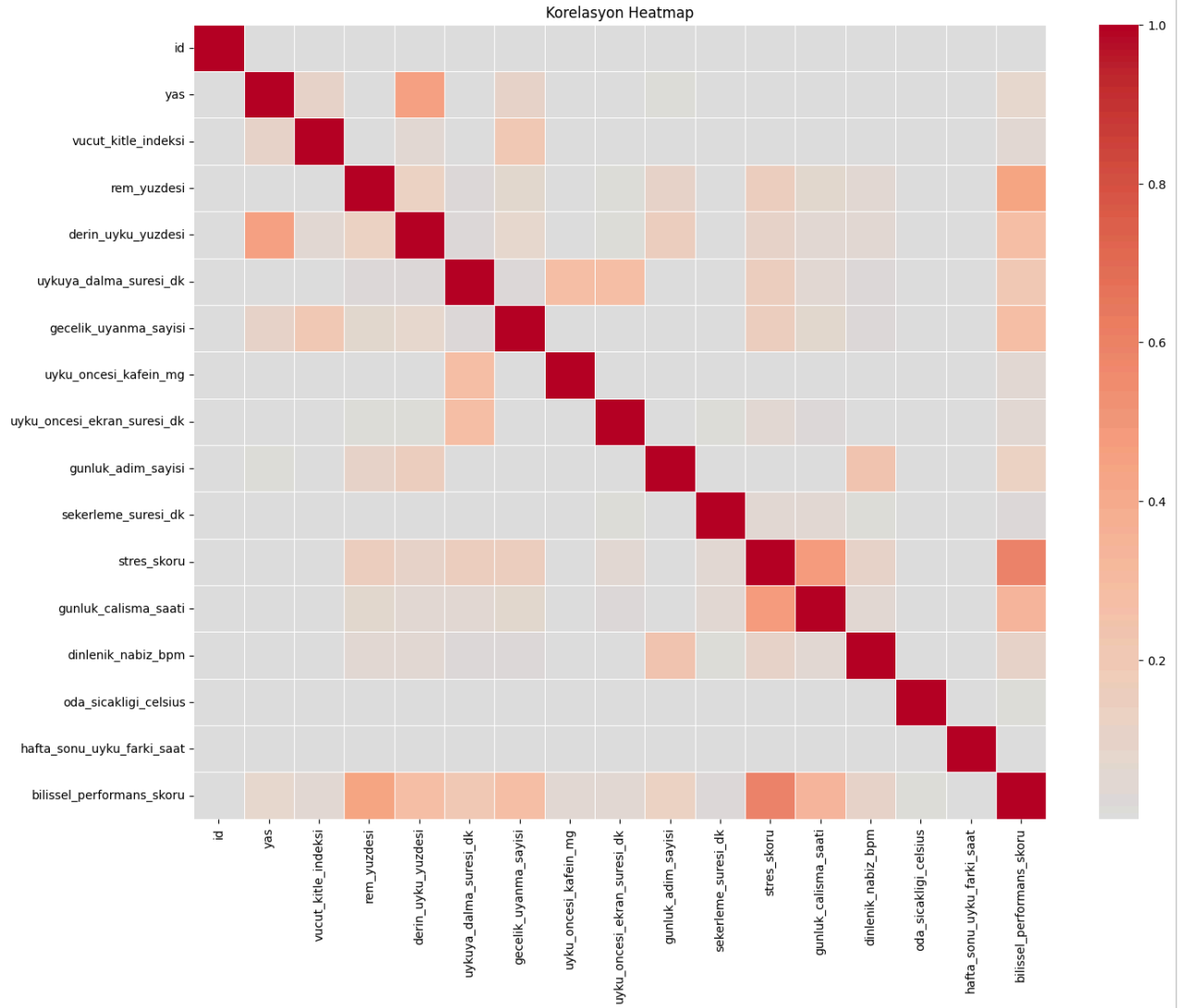


```

        center=0,
        linewidths=0.5
    )

plt.title("Korelasyon Heatmap")
plt.show()

```



8.2 Target Korelasyon Analizi

```
target_corr = corr_matrix[TARGET].sort_values(ascending=False)
```

```
print(target_corr)
```

```

bilissel_performans_skoru    1.000000
stres_skoru                  0.586676
rem_yuzdesi                  0.442999
gunluk_calisma_saati        0.342112
derin_uyku_yuzdesi          0.280006
gecelik_uyanma_sayisi       0.270349
uykuya_dalma_suresi_dk      0.207349
gunluk_adim_sayisi          0.134042
dinlenik_nabiz_bpm          0.089389
yas                          0.070659
uyku_onesi_kafein_mg        0.062387
uyku_onesi_ekran_suresi_dk  0.044631
vucut_kitle_indeksi         0.037990
sekerleme_suresi_dk         0.023683
oda_sicakligi_celsius       0.011992
id                           0.005845

```

9. Aykırı Değer (Outlier) Analizi

9.1 IQR Yöntemi

```
def detect_outliers(df, col):  
  
    Q1 = df[col].quantile(0.25)  
    Q3 = df[col].quantile(0.75)  
  
    IQR = Q3 - Q1  
  
    lower = Q1 - 1.5 * IQR  
    upper = Q3 + 1.5 * IQR  
  
    outliers = df[  
        (df[col] < lower)  
        |  
        (df[col] > upper)  
    ]  
  
    return outliers
```

9.2 Outlier Sayılarının Hesaplanması

```
for col in num_cols:  
  
    outlier_count = len(  
        detect_outliers(train, col)  
    )  
  
    print(col, outlier_count)
```

```
id 0  
yas 184  
vucut_kitle_indeksi 173  
rem_yuzdesi 420  
derin_uyku_yuzdesi 235  
uykuya_dalma_suresi_dk 531  
gecelik_uyanma_sayisi 0  
uyku_onesi_kafein_mg 3360  
uyku_onesi_ekran_suresi_dk 3290  
gunluk_adim_sayisi 166  
sekerleme_suresi_dk 505  
stres_skoru 986  
gunluk_calisma_saati 18  
dinlenik_nabiz_bpm 300  
oda_sicakligi_celsius 0  
hafta_sonu_uyku_farki_saati 120
```

10. Aykırı Değer Yönetimi

10.1 IQR Capping İşlemi

```
for col in num_cols:  
  
    Q1 = train[col].quantile(0.25)  
    Q3 = train[col].quantile(0.75)  
  
    IQR = Q3 - Q1
```

10.2 Alt ve Üst Limit Belirleme

```
lower = Q1 - 1.5 * IQR  
upper = Q3 + 1.5 * IQR
```

10.3 Clip İşlemi ile Baskılama

```
train[col] = train[col].clip(
    lower,
    upper
)

test[col] = test[col].clip(
    lower,
    upper
)
```

11. Eksik Veri Yönetimi

11.1 Kategorik Değişkenlerde Eksik Veri Doldurma

```
cat_fill_cols = [
    "kronotip",
    "meslek",
    "ruh_sagligi_durumu"
]

for col in cat_fill_cols:

    train[col] = train[col].fillna(
        "Unknown"
    )

    test[col] = test[col].fillna(
        "Unknown"
    )
```

11.2 Sayısal Değişkenlerde Median Doldurma

```
num_fill_cols = [
    "vucut_kitle_indeksi",
    "stres_skoru"
]

for col in num_fill_cols:

    median_value = train[col].median()

    train[col] = train[col].fillna(
        median_value
    )

    test[col] = test[col].fillna(
        median_value
    )
```

11.3 Özel Durum Eksik Veri Yönetimi

```
train["uyku_onesi_kafein_mg"] = (
    train["uyku_onesi_kafein_mg"]
    .fillna(0)
)

test["uyku_onesi_kafein_mg"] = (
    test["uyku_onesi_kafein_mg"]
    .fillna(0)
)
```

12. Feature Engineering

12.1 Uyku Kalite Skoru

```
train["uyku_kalite_skoru"] = (
    (
        train["rem_yuzdesi"]
        +
```

```

        train["derin_uyku_yuzdesi"]
    )
    /
    (
        train["gecelik_uyanma_sayisi"] + 1
    )
)

test["uyku_kalite_skoru"] = (
    (
        test["rem_yuzdesi"]
        +
        test["derin_uyku_yuzdesi"]
    )
    /
    (
        test["gecelik_uyanma_sayisi"] + 1
    )
)

```

12.2 Dijital Yorgunluk Skoru

```

train["dijital_yorgunluk"] = (
    train["uyku_onesi_ekran_suresi_dk"]
    *
    train["stres_skoru"]
)

test["dijital_yorgunluk"] = (
    test["uyku_onesi_ekran_suresi_dk"]
    *
    test["stres_skoru"]
)

```

12.3 Beyin Dinlenme Oranı

```

train["beyin_dinlenme_orani"] = (
    (
        train["derin_uyku_yuzdesi"]
        +
        train["rem_yuzdesi"]
    )
    /
    (
        train["stres_skoru"] + 1
    )
)

test["beyin_dinlenme_orani"] = (
    (
        test["derin_uyku_yuzdesi"]
        +
        test["rem_yuzdesi"]
    )
    /
    (
        test["stres_skoru"] + 1
    )
)

```

12.4 Fatigue Index

```

train["fatigue_index"] = (
    train["stres_skoru"]
    *
    train["gecelik_uyanma_sayisi"]
    *
    train["uykuya_dalma_suresi_dk"]
)

test["fatigue_index"] = (

```

```

    test["stres_skoru"]
    *
    test["gecelik_uyanma_sayisi"]
    *
    test["uykuya_dalma_suresi_dk"]
)

```

12.5 Social Jetlag

```

train["social_jetlag"] = (
    train["hafta_sonu_uyku_farki_saat"]
    *
    train["uykuya_dalma_suresi_dk"]
)

test["social_jetlag"] = (
    test["hafta_sonu_uyku_farki_saat"]
    *
    test["uykuya_dalma_suresi_dk"]
)

```

12.6 Kalp Dinlenme Oranı

```

train["kalp_dinlenme"] = (
    train["derin_uyku_yuzdesi"]
    /
    (
        train["dinlenik_nabiz_bpm"] + 1
    )
)

test["kalp_dinlenme"] = (
    test["derin_uyku_yuzdesi"]
    /
    (
        test["dinlenik_nabiz_bpm"] + 1
    )
)

```

12.7 Yaşam Dengesi Skoru

```

train["yasam_dengesi"] = (
    train["gunluk_adim_sayisi"]
    /
    (
        train["gunluk_calisma_saati"]
        +
        train["stres_skoru"] + 1
    )
)

test["yasam_dengesi"] = (
    test["gunluk_adim_sayisi"]
    /
    (
        test["gunluk_calisma_saati"]
        +
        test["stres_skoru"] + 1
    )
)

```

13. Encoding İşlemleri

13.1 Frequency Encoding

```

all_cat_cols = train.select_dtypes(
    include="object"
).columns.tolist()

```

```

for col in all_cat_cols:

    freq = train[col].value_counts()

    train[col + "_freq"] = (
        train[col]
        .map(freq)
    )

    test[col + "_freq"] = (
        test[col]
        .map(freq)
        .fillna(0)
    )

```

13.2 Target Encoding

```

target_cols = [
    "meslek",
    "kronotip",
    "ulke"
]

for col in target_cols:

    target_mean = (
        train.groupby(col)[TARGET]
        .mean()
    )

    train[col + "_target"] = (
        train[col]
        .map(target_mean)
    )

    test[col + "_target"] = (
        test[col]
        .map(target_mean)
    )

```

13.3 Label Encoding

```

for col in all_cat_cols:

    le = LabelEncoder()

    combined = pd.concat([
        train[col],
        test[col]
    ]).astype(str)

    le.fit(combined)

    train[col + "_label"] = le.transform(
        train[col].astype(str)
    )

    test[col + "_label"] = le.transform(
        test[col].astype(str)
    )

```

14. Feature Selection

14.1 Korelasyon Bazlı Feature Elimination

```

corr_matrix = train.corr(numeric_only=True).abs()

```

14.2 Yüksek Korelasyonlu Değişkenlerin Tespiti

```
upper = corr_matrix.where(
    np.triu(
        np.ones(corr_matrix.shape),
        k=1
    ).astype(bool)
)

to_drop = [
    column for column in upper.columns
    if any(upper[column] > 0.95)
]

print(to_drop)

['cinsiyet_label', 'mevsim_label', 'gun_tipi_label']
```

15. Model İçin Veri Hazırlığı

15.1 X ve y Ayrımı

```
X = train.drop(
    ["id", TARGET],
    axis=1
)

y = train[TARGET]
```

15.2 Test Seti Hazırlığı

```
X_test = test.drop(
    ["id"],
    axis=1
)
```

15.3 Kategorik Feature Listesi

```
cat_features = X.select_dtypes(
    include="object"
).columns.tolist()
print(cat_features)

['cinsiyet', 'meslek', 'ulke', 'kronotip', 'ruh_sagligi_durumu', 'mevsim', 'gun_tipi']
```

16. Cross Validation Yapısı

```
kf = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)
```

17. CatBoost Modelinin Oluşturulması

```
model = CatBoostRegressor(
    iterations=5000,
    learning_rate=0.02,
    depth=6,
    l2_leaf_reg=10,
    loss_function="RMSE",
    eval_metric="RMSE",
    random_seed=42,
    early_stopping_rounds=400,
    verbose=500
)
```

18. Model Eğitimi

18.1 Fold Bazlı Eğitim

```

scores = []

oof_preds = np.zeros(len(X))
test_preds = np.zeros(len(X_test))

for fold, (train_idx, valid_idx) in enumerate(kf.split(X)):

    print(f"\n===== FOLD {fold+1} =====")

    X_train = X.iloc[train_idx]
    X_valid = X.iloc[valid_idx]

    y_train = y.iloc[train_idx]
    y_valid = y.iloc[valid_idx]

    model.fit(
        X_train,
        y_train,
        cat_features=cat_features,
        eval_set=(X_valid, y_valid),
        use_best_model=True
    )

2000:   learn: 1.1632502      test: 1.2214887 best: 1.2214249 (1867) total: 1m 30s remaining: 2m 15s
Stopped by overfitting detector (400 iterations wait)

bestTest = 1.221424906
bestIteration = 1867

Shrink model to first 1868 iterations.

===== FOLD 2 =====
0:      learn: 2.2081018      test: 2.1988795 best: 2.1988795 (0) total: 53.9ms remaining: 4m 29s
500:    learn: 1.2096607      test: 1.2245036 best: 1.2245036 (500) total: 23.6s remaining: 3m 31s
1000:   learn: 1.1900336      test: 1.2189120 best: 1.2189082 (989) total: 44.9s remaining: 2m 59s
1500:   learn: 1.1753098      test: 1.2176260 best: 1.2176260 (1500) total: 1m 5s remaining: 2m 33s
2000:   learn: 1.1621249      test: 1.2175643 best: 1.2174594 (1934) total: 1m 28s remaining: 2m 12s
Stopped by overfitting detector (400 iterations wait)

bestTest = 1.217459439
bestIteration = 1934

Shrink model to first 1935 iterations.

===== FOLD 3 =====
0:      learn: 2.2065043      test: 2.2052150 best: 2.2052150 (0) total: 69ms remaining: 5m 44s
500:    learn: 1.2130087      test: 1.2093143 best: 1.2093143 (500) total: 23.9s remaining: 3m 34s
1000:   learn: 1.1944466      test: 1.2039851 best: 1.2039836 (996) total: 45.8s remaining: 3m 3s
1500:   learn: 1.1804723      test: 1.2032709 best: 1.2032382 (1470) total: 1m 7s remaining: 2m 38s
2000:   learn: 1.1674446      test: 1.2033779 best: 1.2031685 (1773) total: 1m 29s remaining: 2m 14s
Stopped by overfitting detector (400 iterations wait)

bestTest = 1.203168451
bestIteration = 1773

Shrink model to first 1774 iterations.

===== FOLD 4 =====
0:      learn: 2.2082216      test: 2.1981730 best: 2.1981730 (0) total: 50.1ms remaining: 4m 10s
500:    learn: 1.2110605      test: 1.2157117 best: 1.2157117 (500) total: 24s remaining: 3m 35s
1000:   learn: 1.1893039      test: 1.2105691 best: 1.2105547 (993) total: 46.9s remaining: 3m 7s
1500:   learn: 1.1751641      test: 1.2095778 best: 1.2095778 (1500) total: 1m 8s remaining: 2m 40s
Stopped by overfitting detector (400 iterations wait)

bestTest = 1.209575112
bestIteration = 1504

Shrink model to first 1505 iterations.

===== FOLD 5 =====
0:      learn: 2.2042902      test: 2.2138825 best: 2.2138825 (0) total: 65.6ms remaining: 5m 28s
500:    learn: 1.2059625      test: 1.2421545 best: 1.2421545 (500) total: 23.7s remaining: 3m 32s
1000:   learn: 1.1887131      test: 1.2353651 best: 1.2353651 (1000) total: 43.9s remaining: 2m 55s
1500:   learn: 1.1741694      test: 1.2338500 best: 1.2338430 (1495) total: 1m 5s remaining: 2m 32s
2000:   learn: 1.1596343      test: 1.2332338 best: 1.2332144 (1996) total: 1m 28s remaining: 2m 12s
Stopped by overfitting detector (400 iterations wait)

bestTest = 1.233180045
bestIteration = 2088

Shrink model to first 2089 iterations.

```



```
preds = model.predict(X_valid)
```

18.3 RMSE Hesaplama

```
rmse = np.sqrt(
    mean_squared_error(y_valid, preds)
)
print("Fold RMSE:", rmse)

scores.append(rmse)
```

```
Fold RMSE: 1.2331800452521735
```

18.4 Out Of Fold Prediction

```
oof_preds[valid_idx] = preds
```

18.5 Test Prediction

```
test_preds += model.predict(X_test) / 5
```

19. Final Model Performansı

19.1 Fold RMSE Sonuçları

```
print(scores)
```

```
[np.float64(1.2331800452521735)]
```

19.2 Ortalama RMSE Hesaplama

```
print("\n=====")
print("MEAN RMSE:", np.mean(scores))
print("=====")
```

```
=====
MEAN RMSE: 1.2331800452521735
=====
```

20. Feature Importance Analizi

20.1 Importance DataFrame Oluşturulması

```
importance_df = pd.DataFrame({
    "feature": X.columns,
    "importance": model.feature_importances_
})
```

20.2 En Önemli Özelliklerin Analizi

```
importance_df = (
    importance_df
    .sort_values(
        by="importance",
        ascending=False
    )
)

print(importance_df.head(20))
```

	feature	importance
24	beyin_dinlenme_orani	26.630067
43	ruh_sagligi_durumu_label	11.413130
21	gun_tipi	7.671439

5	rem_yuzdesi	7.337833
36	meslek_target	6.464406
16	ruh_sagliigi_durumu	5.240728
22	uyku_kalite_skoru	4.878873
25	fatigue_index	4.313601
35	gun_tipi_freq	3.637683
45	gun_tipi_label	3.514964
33	ruh_sagliigi_durumu_freq	3.506939
28	yasam_dengesi	1.620981
13	stres_skoru	1.611945
18	oda_sicakligi_celsius	1.221676
2	meslek	1.146577
11	gunluk_adim_sayisi	1.128396
12	sekerleme_suresi_dk	0.967232
7	uykuya_dalma_suresi_dk	0.820139
9	uyku_onesi_kafein_mg	0.802942
6	derin_uyku_yuzdesi	0.738066

20.3 Feature Importance Görselleştirme

```
plt.figure(figsize=(12,8))

sns.barplot(

    data=importance_df.head(20),

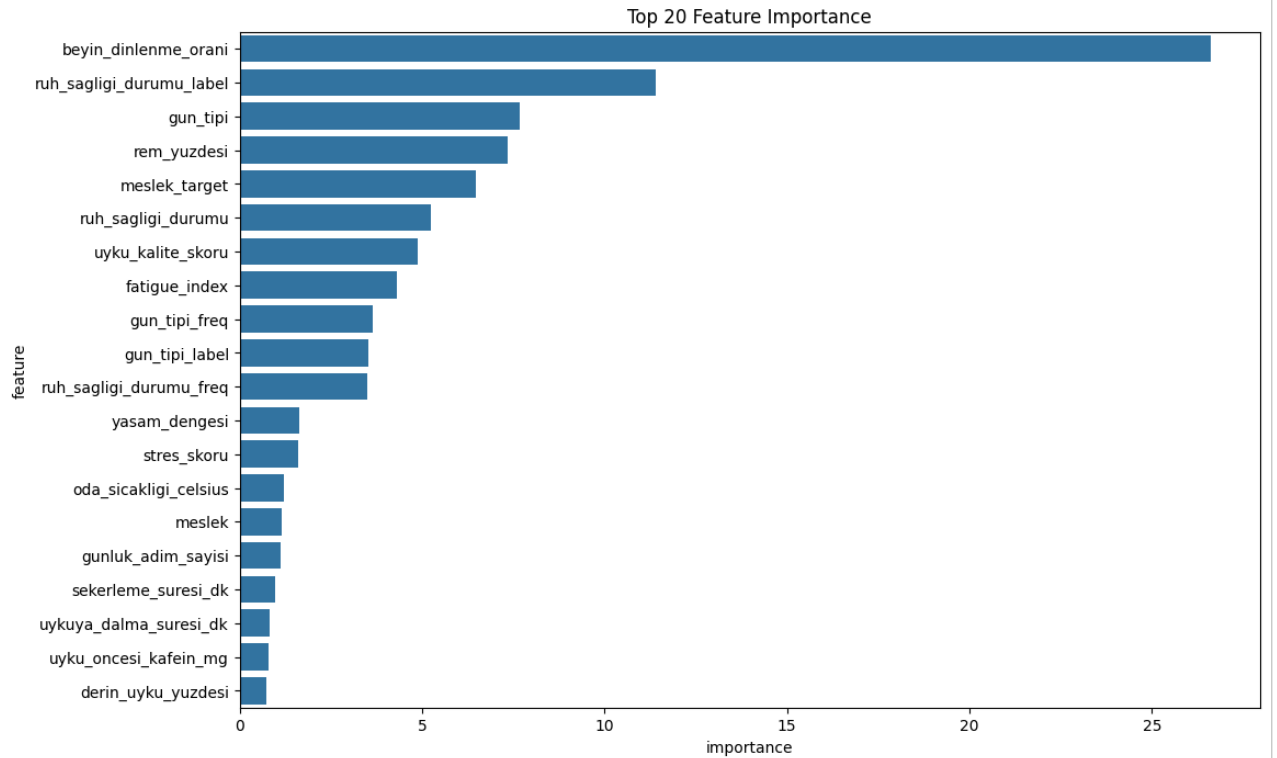
    x="importance",

    y="feature"

)

plt.title("Top 20 Feature Importance")

plt.show()
```



21. SHAP Analizi

21.1 SHAP Explainer

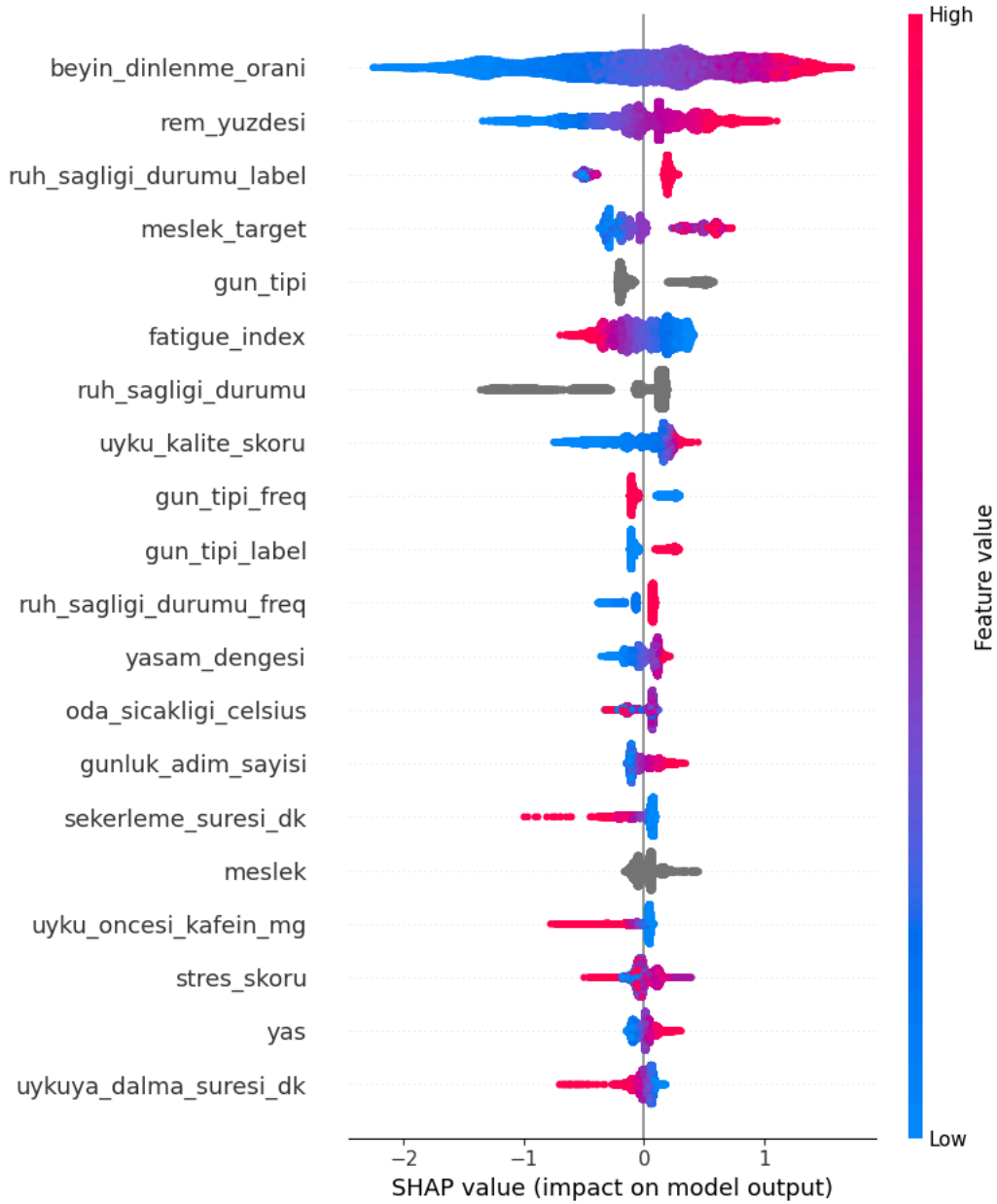
```
explainer = shap.TreeExplainer(model)
```

21.2 SHAP Value Hesaplama

```
shap_values = explainer.shap_values(X)
```

21.3 SHAP Summary Plot

```
shap.summary_plot(  
    shap_values,  
    X  
)
```



22. Hyperparameter Optimization

22.1 Optuna Kurulumu

```
!pip install optuna -q  
  
import optuna
```

22.2 Objective Function

```
def objective(trial):  
    params = {
```

```

        "iterations": 1000,

        "learning_rate": trial.suggest_float(
            "learning_rate",
            0.01,
            0.05
        ),

        "depth": trial.suggest_int(
            "depth",
            4,
            10
        ),

        "l2_leaf_reg": trial.suggest_int(
            "l2_leaf_reg",
            1,
            20
        ),

        "random_strength": trial.suggest_float(
            "random_strength",
            0.1,
            5
        ),

        "loss_function": "RMSE",

        "eval_metric": "RMSE",

        "verbose": 0
    }

    scores = []

    for train_idx, valid_idx in kf.split(X):

        X_train = X.iloc[train_idx]
        X_valid = X.iloc[valid_idx]

        y_train = y.iloc[train_idx]
        y_valid = y.iloc[valid_idx]

        model = CatBoostRegressor(**params)

        model.fit(
            X_train,
            y_train,
            cat_features=cat_features,
            eval_set=(X_valid, y_valid),
            use_best_model=True,
            early_stopping_rounds=100
        )

        preds = model.predict(X_valid)

        rmse = np.sqrt(
            mean_squared_error(y_valid, preds)
        )

        scores.append(rmse)

    return np.mean(scores)

```

22.3 Parametre Arama Uzayı

```

study = optuna.create_study(
    direction="minimize"
)

```

[I 2026-05-17 15:20:40,488] A new study created in memory with name: no-name-fe73fbf8-1566-4494-ba09-9c636236c340

22.4 Best Parameter Seçimi

```
study.optimize(
    objective,
    n_trials=20
)

print(study.best_params)

print(study.best_value)
```

```
[I 2026-05-17 15:25:40,233] Trial 0 finished with value: 1.221764927703895 and parameters: {'learning_rate': 0.0139129435562
[I 2026-05-17 15:28:41,713] Trial 1 finished with value: 1.2179018612414076 and parameters: {'learning_rate': 0.031263547343
[I 2026-05-17 15:32:07,641] Trial 2 finished with value: 1.2187327356330946 and parameters: {'learning_rate': 0.048072527259
[I 2026-05-17 15:35:12,630] Trial 3 finished with value: 1.2178778476857888 and parameters: {'learning_rate': 0.028549878021
[I 2026-05-17 15:47:39,708] Trial 4 finished with value: 1.2235841958527576 and parameters: {'learning_rate': 0.012805658548
[I 2026-05-17 15:54:09,805] Trial 5 finished with value: 1.2186745513996395 and parameters: {'learning_rate': 0.018314419328
[I 2026-05-17 15:58:04,921] Trial 6 finished with value: 1.2187355921175649 and parameters: {'learning_rate': 0.018113354137
[I 2026-05-17 16:04:25,317] Trial 7 finished with value: 1.2183849062186103 and parameters: {'learning_rate': 0.024396611683
[I 2026-05-17 16:09:23,475] Trial 8 finished with value: 1.219965046722648 and parameters: {'learning_rate': 0.0148620109322
[I 2026-05-17 16:14:32,668] Trial 9 finished with value: 1.2185531730030148 and parameters: {'learning_rate': 0.038674248605
[I 2026-05-17 16:16:52,787] Trial 10 finished with value: 1.2184403464588245 and parameters: {'learning_rate': 0.03295166605
[I 2026-05-17 16:19:13,191] Trial 11 finished with value: 1.2189382097572763 and parameters: {'learning_rate': 0.02995154432
[I 2026-05-17 16:22:08,856] Trial 12 finished with value: 1.2173096469960636 and parameters: {'learning_rate': 0.03681939002
[I 2026-05-17 16:25:06,566] Trial 13 finished with value: 1.217432097318637 and parameters: {'learning_rate': 0.039382886906
[I 2026-05-17 16:28:04,278] Trial 14 finished with value: 1.2177667616509087 and parameters: {'learning_rate': 0.04182032491
[I 2026-05-17 16:31:45,713] Trial 15 finished with value: 1.217047546827835 and parameters: {'learning_rate': 0.039846399175
[I 2026-05-17 16:35:00,372] Trial 16 finished with value: 1.2177071744898542 and parameters: {'learning_rate': 0.04946961667
[I 2026-05-17 16:38:27,893] Trial 17 finished with value: 1.218228770908614 and parameters: {'learning_rate': 0.043008218397
[I 2026-05-17 16:45:38,452] Trial 18 finished with value: 1.2194360260424282 and parameters: {'learning_rate': 0.03476078208
[I 2026-05-17 16:47:54,732] Trial 19 finished with value: 1.2179413567812358 and parameters: {'learning_rate': 0.04527075506
{'learning_rate': 0.03984639917580456, 'depth': 6, 'l2_leaf_reg': 14, 'random_strength': 4.014525336842285}
1.217047546827835
```

23. Ensemble Learning

23.1 LightGBM Modeli

```
lgbm_model = LGBMRegressor(

    n_estimators=3000,

    learning_rate=0.02,

    max_depth=6,

    random_state=42

)

X_train_lgbm = X_train.drop(columns=cat_features)
X_test_lgbm = X_test.drop(columns=cat_features)

lgbm_model.fit(X_train_lgbm, y_train)

lgbm_preds = lgbm_model.predict(X_test_lgbm)
```

[Gizli çıkışı göster](#)

23.2 XGBoost Modeli

```
xgb_model = XGBRegressor(

    n_estimators=3000,

    learning_rate=0.02,

    max_depth=6,

    random_state=42

)

X_train_xgb = X_train.drop(columns=cat_features)
X_test_xgb = X_test.drop(columns=cat_features)
```

```
xgb_model.fit(X_train_xgb, y_train)

xgb_preds = xgb_model.predict(X_test_xgb)
```

23.3 Ensemble Prediction

```
cat_preds = test_preds
```

23.4 Weighted Averaging

```
final_preds = (

    0.5 * cat_preds +

    0.3 * lgbm_preds +

    0.2 * xgb_preds

)
```

24. Residual Analizi

24.1 Residual Hesaplama

```
residuals = y - oof_preds
```

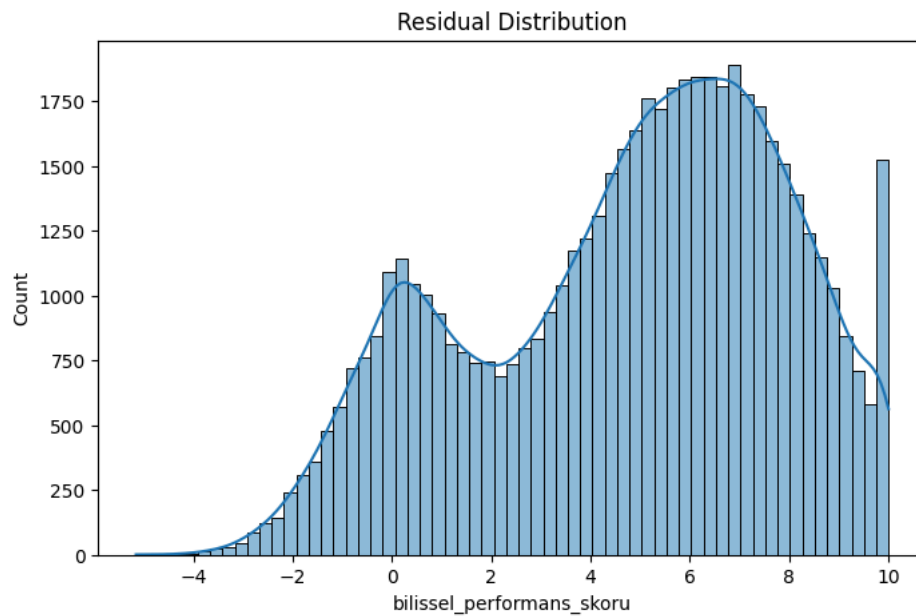
24.2 Residual Distribution Görselleştirme

```
plt.figure(figsize=(8,5))

sns.histplot(
    residuals,
    kde=True
)

plt.title("Residual Distribution")

plt.show()
```



25. Submission Dosyasının Oluşturulması

25.1 Submission DataFrame

```
submission = pd.DataFrame({  
    "id": test["id"],  
    TARGET: final_preds  
})
```

25.2 CSV Export işlemi

```
submission.to_csv(  
    "submission.csv",  
    index=False  
)  
  
print("submission.csv oluşturuldu.")  
  
submission.csv oluşturuldu.
```